COMPLETE AI, MACHINE LEARNING & AZURE PRACTICE PROBLEM BOOK

Azure ML • Advanced ML • Deep Learning • NLP • OCR • Custom Vision • GenAI • Microsoft Foundry • Bots • Responsible AI • Deployment • APIs • MLOps

How to Use This Practice Book

This document consolidates the hands-on exercises, scenarios, modeling tasks, architecture problems and assessment questions developed across the training topics.

- Attempt each problem before consulting documentation.
- For Azure labs, record resource names, model versions, endpoint names and results.
- For ML projects, report metrics and explain limitations.
- For production tasks, include security, monitoring and failure handling.

1. Azure Machine Learning – Foundations & Setup

Problem 1 — Create an Azure ML Workspace

Create a workspace, open it in Studio, identify associated resources, record region/name, and explain the workspace's role.

Problem 2 — Compute Target Selection

Choose suitable compute for a 100,000-row sklearn workload and a CNN image workload. Justify CPU vs GPU and managed vs local compute.

Problem 3 — Create an Environment

Create an environment containing Python, scikit-learn, pandas, numpy and MLflow. Remove a dependency intentionally, observe the failure, restore it and rerun.

Problem 4 — Dataset/Data Asset Management

Register a public CSV dataset, create a second version, compare schema/row count, train models from both versions and record lineage.

Problem 5 — Experiment Tracking

Train five configurations, log parameters/metrics/artifacts, identify the best run and explain why the best metric alone may not determine production choice.

Problem 6 — Azure ML Designer

Build a classification pipeline: dataset → preprocessing → split → train → score → evaluate; compare two algorithms.

2. Advanced Machine Learning Techniques

Problem 7 — Feature Engineering

Using a churn dataset, create at least five engineered features such as tenure groups, average spend, lifetime value and activity ratios. Compare against baseline.

Problem 8 — Feature Selection

Use correlation analysis and SelectKBest or another sklearn selector. Compare performance and training time.

Problem 9 — GridSearchCV

Tune a RandomForestClassifier with n_estimators, max_depth and min_samples_split. Report best parameters and metrics.

Problem 10 — RandomizedSearchCV

Use broader parameter distributions, sample at least 20 configurations, compare runtime and model quality with GridSearchCV.

Problem 11 — Ensemble Methods

Compare BaggingClassifier, Random Forest, Gradient Boosting and VotingClassifier using the same split and metrics.

Problem 12 — Complete Advanced ML Challenge

Build a full sklearn pipeline with cleaning, feature engineering, feature selection, baseline, GridSearchCV, RandomizedSearchCV, ensembles, cross-validation and final model selection.

3. Deep Learning with Azure ML Compute

Problem 13 — ANN Classification

Build an ANN for binary classification, scale inputs, train, plot loss, evaluate and explain overfitting.

Problem 14 — CNN Image Classification

Organize image classes, create train/validation/test sets, normalize, train a CNN, create a confusion matrix and inspect errors.

Problem 15 — RNN/LSTM Sequence Problem

Build an RNN/LSTM for sentiment or time-series prediction; compare behavior and discuss sequence-learning limitations.

Problem 16 — GPU Training in Azure ML

Train a CNN on Azure GPU compute and compare CPU vs GPU training time and resource usage.

Problem 17 — Transfer Learning

Use a pretrained image model, freeze the backbone, train a new head, unfreeze selected layers and fine-tune with a lower learning rate.

Problem 18 — Model Export & Deployment

Save a TensorFlow/PyTorch model, package preprocessing, create inference code, test locally and deploy where supported.

4. Natural Language Processing

Problem 19 — Text Preprocessing

Implement lowercasing, tokenization, stop-word removal, punctuation handling and stemming/lemmatization on customer reviews.

Problem 20 — TF-IDF Sentiment Model

Build TF-IDF + Logistic Regression; evaluate accuracy, precision, recall and F1; inspect important features.

Problem 21 — Word Embeddings

Compare TF-IDF with dense word/sentence embeddings and discuss semantic similarity.

Problem 22 — Transformer Sentiment Analysis

Use a pretrained transformer for sentiment; compare with the TF-IDF baseline and analyze disagreements and latency.

Problem 23 — Named Entity Recognition

Extract people, organizations, locations and dates from at least 20 sentences and create an entity table.

Problem 24 — Complete NLP Project

Create an app that accepts a customer review and returns sentiment, important keywords and named entities.

5. OCR and Document Understanding

Problem 25 — OCR Invoice Extraction

Extract invoice number, date, vendor/customer and total amount. Compare with ground truth and calculate field-level accuracy.

Problem 26 — OCR Quality Challenge

Test clean, blurred, rotated, low-light and complex-background documents. Compare extraction quality.

Problem 27 — Document Understanding

Design a loan-application pipeline with OCR, document classification, field extraction, validation, human review and audit trail.

6. Custom Vision

Problem 28 — Defective Product Classification

Dataset: NEU Surface Defect Database — https://www.kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database

1. Create a Custom Vision image-classification project.
2. Prepare labels and upload images.
3. Train and evaluate precision/recall.
4. Test unseen images.
5. Suggest three improvement strategies.

Problem 29 — Hard Hat Object Detection

Dataset: Hard Hat Workers — https://www.kaggle.com/datasets/andrewmvd/hard-hat-detection

6. Create an object-detection project.
7. Upload images and create bounding boxes.
8. Train and test.
9. Inspect false positives/false negatives.
10. Improve the model using varied training images.

Problem 30 — Precision vs Recall

A helmet detector reports precision = 95% and recall = 62%. Explain both metrics, diagnose missed detections, propose five ways to improve recall and retrain.

7. Generative AI & Azure OpenAI

Problem 31 — Prompt Engineering

Improve the prompt 'Analyze this sales data and tell me what happened.' Include role, task, context, output format and constraints; test on multiple examples.

Problem 32 — Structured Output

Create a prompt that converts customer complaints into Category, Severity, Customer issue, Recommended action and Escalation required.

Problem 33 — Grounded Business Assistant

Build a document-grounded assistant that answers only from supplied company policy, avoids invention and states when evidence is unavailable.

Problem 34 — Responsible GenAI Testing

Create tests for hallucination, prompt injection and inappropriate data exposure; define expected safe behavior.

Problem 35 — Fine-Tuning Decision

Decide between prompt engineering, few-shot examples, RAG and fine-tuning for a specialized output requirement. Justify based on freshness, consistency, cost and maintenance.

8. Microsoft Foundry & Prompt Engineering

Problem 36 — Finance Assistant Agent

Dataset: Financial Statements — https://www.kaggle.com/datasets/atharvaingle/financial-statements

- Analyze financial documents.

- Summarize statements.
- Answer revenue/expense questions.
- Calculate basic metrics.
- Refuse unauthorized financial actions.

Define instructions, knowledge sources, tools, safety rules and expected responses.

Problem 37 — Customer Review Prompt

Dataset: Amazon Reviews — https://www.kaggle.com/datasets/bittlingmayer/amazonreviews

Create a prompt returning Sentiment, Main complaint, Main positive point, Product issue and Overall summary. Test 10 reviews.

Problem 38 — Model Evaluation

Model	Accuracy	Response Time	Cost
A	94%	1.2 sec	High
B	91%	0.5 sec	Low

Requirement: accuracy ≥ 90%, low latency and low cost. Select a model, test both on common inputs, evaluate consistency/errors/hallucinations, measure latency and recommend production choice.

Problem 39 — Customer Support Agent

Dataset: CLINC150 — https://www.kaggle.com/datasets/hassanamin/clinc150

- Understand questions.
- Use knowledge/RAG.
- Call tools/APIs.
- Create support tickets.
- Escalate complex cases.
- Maintain context.
- Protect confidential customer information.

9. Azure Bot Framework Composer

Problem 40 — College Admission Bot

Dataset: Indian College Admission Dataset — https://www.kaggle.com/datasets/akshaydattatraykhare/indian-college-admission-dataset

Create a bot answering courses, eligibility, fees and admission requirements. Use dialogs, triggers, variables, conditions, Adaptive Expressions and user input.

Problem 41 — Eligibility Condition

```
Age >= 18 → Eligible
Age < 18  → Not eligible
```

Implement the condition in Composer and extend it with course, qualification and percentage. Add invalid-input handling.

Problem 42 — Weather API Bot

Resource: Open-Meteo — https://open-meteo.com/

Capture city, call the weather API, store the response, extract temperature and display it. Add API-failure and invalid-city handling.

10. Responsible AI & Explainability

Problem 43 — Bias Detection

11. Check class and group distributions.
12. Check label rates.
13. Calculate overall metrics.
14. Calculate group-level selection rate, TPR and FPR where appropriate.
15. Identify disparities.
16. Explain why disparity alone does not prove bias.
17. Propose further analysis.

Problem 44 — Responsible AI Dashboard

18. Inspect data representation.
19. Analyze errors.
20. Compare relevant group performance.
21. Inspect interpretability.
22. Investigate a high-error cohort.
23. Propose mitigation.
24. Re-evaluate.

Problem 45 — Privacy & Compliance

A healthcare organization wants to train on patient records. Identify personal/sensitive data, minimization, access controls, de-identification, retention, output risks, human oversight and audit requirements.

11. Model Registration, Export & Deployment

Problem 46 — Register a Model

25. Train a sklearn model.
26. Log metrics.
27. Save/log the artifact.
28. Register it in Azure ML.
29. Add description/tags.
30. Record version.
31. Compare with another version.

Problem 47 — Model Export

Export using MLflow, joblib or ONNX where supported. Identify dependencies, package preprocessing, load in a clean environment and verify predictions.

Problem 48 — Managed Online Endpoint

32. Create endpoint.
33. Create environment.
34. Create scoring script.
35. Deploy registered model.
36. Check health.
37. Send JSON inference request.
38. Test invalid input.
39. Inspect logs.
40. Record endpoint/model version.

Problem 49— Batch vs Online Architecture

Choose and justify online or batch inference for: transaction fraud, nightly scoring for 2 million customers, real-time recommendations and monthly risk reports.

12. Real-Time AI Applications & APIs

Problem 50 — FastAPI Model API

```
POST /predict
Request:  {"features": [...]}
Response: {"prediction": 1, "probability": 0.91}
```

- Build the API around a model.
- Add Pydantic validation.
- Handle invalid input.
- Return appropriate status codes.
- Document and test with requests.

13. Dataset & Resource List

Dataset / Resource	Use	Link
NEU Surface Defect Database	Custom Vision	https://www.kaggle.com/datasets/kaustubhdikshit/neu-surface-defect-database
Hard Hat Workers	Custom Vision	https://www.kaggle.com/datasets/andrewmvd/hard-hat-detection
Indian College Admission	Bot Composer	https://www.kaggle.com/datasets/akshaydattatraykhare/indian-college-admission-dataset
Financial Statements	Foundry Finance Agent	https://www.kaggle.com/datasets/atharvaingle/financial-statements
Amazon Reviews	Prompt engineering/NLP	https://www.kaggle.com/datasets/bittlingmayer/amazonreviews
CLINC150	Customer support/intent	https://www.kaggle.com/datasets/hassanamin/clinc150
Open-Meteo	Bot/API integration	https://open-meteo.com/

14. Final Viva / Discussion Questions

- Why is a model registry different from an endpoint?
- Why should preprocessing be packaged with a model?
- When would you choose GridSearchCV over RandomizedSearchCV?
- Why might a GPU not improve every ML workload?
- What is the difference between TF-IDF and embeddings?
- When should RAG be preferred over fine-tuning?
- What is the difference between an agent and a simple chatbot?
- Why does removing a sensitive attribute not guarantee fairness?
- What do SHAP and LIME explain, and what do they not prove?
- When should you use batch rather than online inference?
- What causes training-serving skew?
- Why do real-time systems need queues?
- What is consumer lag?
- Why is P95 latency important?
- Why is model monitoring different from API monitoring?
- What is data drift?
- What is concept drift?
- What is a champion/challenger workflow?
- Why should retraining not automatically mean production deployment?
- What should a rollback strategy contain?